UNIVERSIDAD DE LAS FUERZAS ARMADAS-ESPE - MATRIZ

DEPARTAMENTO DE CIENCIAS DE LA COMPUTACIÓN - DCCO-SS

CARRERA DE INGENIERÍA EN TECNOLOGÍAS DE LA INFORMACIÓN

ASIGNATURA : MODELOS DISCRETOS PARA INGENIERÍA

TEMA : Grafos

ESTUDIANTE : Bryan Cevallos

NIVEL-PARALELO - NRC: Tercero A – NRC : **29030**

DOCENTE : ING. WASHINGTON LOZA

FECHA DE ENTREGA : 13/01/2026.

SANGOLQUI – ECUADOR

Código

Clase Grafo :

```
#include <iostream>
#include <vector>
using namespace std;

// Estructura para representar un segmento (arista)
struct Segmento {
    int nodo1;
    int nodo2;
};

class Grafo {
private:
    int cantidadNodos;
    int cantidadSegmentos;
    vector<Segmento> segmentos;      // Lista de segmentos
    vector<int> grados;              // Grado de cada nodo

public:

    Grafo() {
        cantidadNodos = 0;
        cantidadSegmentos = 0;
    }

    void ingresarNodos() {
        cout << "Ingrese la cantidad de nodos: ";
        cin >> cantidadNodos;
        grados.resize(cantidadNodos, 0);
    }
    void ingresarSegmentos() {
        cout << "Ingrese la cantidad de segmentos: ";
        cin >> cantidadSegmentos;

        cout << "\nIngrese los segmentos (nodos que se unen):\n";
        cout << "Nota: Los nodos van de 0 a " << (cantidadNodos - 1) << "\n\n";

        for (int i = 0; i < cantidadSegmentos; i++) {
            Segmento seg;
            cout << "Segmento " << (i + 1) << ":\n";
            cout << "  Nodo 1: ";
            cin >> seg.nodo1;
            cout << "  Nodo 2: ";
```

```
        cin >> seg.nodo2;

        // Validar que los nodos existan
        if (seg.nodo1 < 0 || seg.nodo1 >= cantidadNodos ||
            seg.nodo2 < 0 || seg.nodo2 >= cantidadNodos) {
            cout << "  ERROR: Nodo invalido. Intente de nuevo.\n";
            i--;
            continue;
        }
        segmentos.push_back(seg);
        grados[seg.nodo1]++;
        grados[seg.nodo2]++;
    }
}
void mostrarSegmentos() {
    cout << "\n=== SEGMENTOS DEL GRAFO ===\n";
    for (int i = 0; i < segmentos.size(); i++) {
        cout << "Segmento " << (i + 1) << ": ("
            << segmentos[i].nodo1 << ", "
            << segmentos[i].nodo2 << ")\n";
    }
}
void mostrarGrados() {
    cout << "\n=== GRADOS DE LOS NODOS ===\n";
    for (int i = 0; i < cantidadNodos; i++) {
        cout << "Nodo " << i << ": grado = " << grados[i] << "\n";
    }
}
void mostrarMatrizIncidencia() {
    cout << "\n=== MATRIZ DE INCIDENCIA ===\n";
    cout << "Filas = Nodos, Columnas = Segmentos\n\n";


    cout << "    ";
    for (int j = 0; j < cantidadSegmentos; j++) {
        cout << "S" << j << " ";
    }
    cout << "\n";

    cout << "   +";
    for (int j = 0; j < cantidadSegmentos; j++) {
        cout << "---";
    }
    cout << "\n";


    for (int i = 0; i < cantidadNodos; i++) {
```

```
            cout << "N" << i << " | ";


            for (int j = 0; j < cantidadSegmentos; j++) {

                if (segmentos[j].nodo1 == i || segmentos[j].nodo2 == i) {
                    cout << "1  ";
                }
                else {
                    cout << "0  ";
                }
            }
            cout << "\n";
        }
    }
    int calcularValencia() {
        int valencia = 0;
        for (int i = 0; i < cantidadNodos; i++) {
            valencia += grados[i];
        }
        return valencia;
    }
    void mostrarValencia() {
        int valencia = calcularValencia();
        cout << "\n=== VALENCIA DEL GRAFO ===\n";
        cout << "Valencia (suma de grados) = " << valencia << "\n";
        cout << "Calculo: ";
        for (int i = 0; i < cantidadNodos; i++) {
            cout << grados[i];
            if (i < cantidadNodos - 1) cout << " + ";
        }
        cout << " = " << valencia << "\n";
    }
    void mostrarResumen() {
        cout << "\n========================================\n";
        cout << "          RESUMEN DEL GRAFO          \n";
        cout << "========================================\n";
        cout << "Cantidad de nodos: " << cantidadNodos << "\n";
        cout << "Cantidad de segmentos: " << cantidadSegmentos << "\n";
        mostrarSegmentos();
        mostrarMatrizIncidencia();
        mostrarGrados();
        mostrarValencia();

        cout << "========================================\n";
    }
};
```

Clase Main :

```
#include <iostream>
#include <vector>
#include "Grafo.h"
using namespace std;

// Función principal
int main() {
    cout << "========================================\n";
    cout << "  CALCULADOR DE GRADOS Y VALENCIA     \n";
    cout << "========================================\n\n";
    Grafo grafo;
    // Paso 1: Ingresar nodos
    grafo.ingresarNodos();

    // Paso 2: Ingresar segmentos
    grafo.ingresarSegmentos();
    // Paso 3: Mostrar resultados
    grafo.mostrarResumen();
    return 0;
}
```

Capturas de pantalla

```
========================================
  CALCULADOR DE GRADOS Y VALENCIA
========================================

Ingrese la cantidad de nodos: 4
Ingrese la cantidad de segmentos: 7

Ingrese los segmentos (nodos que se unen):
Nota: Los nodos van de 0 a 3

Segmento 1:
  Nodo 1: 0
  Nodo 2: 1
Segmento 2:
  Nodo 1: 0
  Nodo 2: 1
Segmento 3:
  Nodo 1: 1
  Nodo 2: 2
Segmento 4:
  Nodo 1: 2
  Nodo 2: 3
Segmento 5:
  Nodo 1: 3
  Nodo 2: 0
Segmento 6:
  Nodo 1: 3
  Nodo 2: 3
Segmento 7:
  Nodo 1: 0
  Nodo 2: 2
```

```
========================================
            RESUMEN DEL GRAFO
========================================
Cantidad de nodos: 4
Cantidad de segmentos: 7

=== SEGMENTOS DEL GRAFO ===
Segmento 1: (0, 1)
Segmento 2: (0, 1)
Segmento 3: (1, 2)
Segmento 4: (2, 3)
Segmento 5: (3, 0)
Segmento 6: (3, 3)
Segmento 7: (0, 2)

=== MATRIZ DE INCIDENCIA ===
Filas = Nodos, Columnas = Segmentos

    S0 S1 S2 S3 S4 S5 S6
   +--------------------
N0 | 1  1  0  0  1  0  1
N1 | 1  1  1  0  0  0  0
N2 | 0  0  1  1  0  0  1
N3 | 0  0  0  1  1  1  0

=== GRADOS DE LOS NODOS ===
Nodo 0: grado = 4
Nodo 1: grado = 3
Nodo 2: grado = 3
Nodo 3: grado = 4

=== VALENCIA DEL GRAFO ===
Valencia (suma de grados) = 14
Calculo: 4 + 3 + 3 + 4 = 14
========================================
```